GEOGUESSR AI • ENGINEERING HANDBOOK

Global & Hyper-Local Vision Locator Complete Beginner & Team Playbook

A clear, practical guide explaining vision AI architecture, spatial Test-Time Augmentation, active learning memory, and a structured 4-person collaborative engineering workflow.

PROJECT BRIEF & EXECUTIVE SUMMARY

This project bridges global street-level geolocalization and hyper-local village navigation (Kerala micro-regions, Pan-India states, and international spots). Built with **StreetCLIP**, an identity-initialized **Residual Vision Adapter**, multi-scale **Test-Time Augmentation (TTA)**, and an interactive Streamlit UI featuring **Google Earth 3D**, **Google Maps**, and **Esri World Satellite Imagery**. This document details how every component works and outlines the standard operating procedures for a **4-person engineering team**.

Authors & Engineering:	Team of 4 (Location Curator, Image Collector, Trainer, Developer)
Vision Foundation Backbone:	StreetCLIP (geolocal/StreetCLIP • 1.7GB ViT-B/32)
Vision Adapter Architecture:	Residual Bottleneck MLP [768 → 256 → GELU → 768] (alpha=0.5)
Location Knowledge Base:	202+ Canonical Places (Global, Pan-India States & UTs, Kerala Districts)
Adapted Dataset Accuracy:	100% Top-1 Accuracy on evaluation dataset (Zero-Shot baseline: 66.04%)
Version & Environment:	v2.2 Production Ready • Python 3.14 • PyTorch • Streamlit

Table of Contents:

- 1. Conceptual Foundation: How AI Predicts Location from a Photo
- 2. Technical Architecture: StreetCLIP, Residual Adapter & Multi-Scale TTA
- 3. Active Learning & Strict Duplicate Prevention
- 4. The 4-Person Team Roles & Operational Playbook
- 5. Git & GitHub Workflow for Beginners (Zero-Conflict Strategy)
- 6. Quality Gate: The Pre-Flight Automated Validation Script
- 7. Troubleshooting, Best Practices & Quick Reference

1. Conceptual Foundation: How AI Predicts Location from a Photo

When expert GeoGuessr players examine an unfamiliar street view photo, they search for microscopic visual signals: the side cars drive on, lane marking styles, soil pigmentation, vegetation species, telephone pole construction, and architectural patterns. Our AI model replicates this human detective intuition mathematically.

The GeoGuessr Visual Clue Hierarchy:

Signal Category	Visual Characteristics	Geographic Significance
Driving Side	Left vs. Right lane traffic, driver seat position	Left: India, UK, Japan, Australia, South Africa. Right: US, Europe, China, Middle East.
Road & Markings	Yellow dashed lines, white outer borders, asphalt texture	Double solid white/yellow = US/Europe. Weathered single yellow = Kerala / Indian state highways.
Soil & Geology	Red laterite, black regur, pale sand, rocky cliffs	Red soil: Kerala midlands, Goa, Brazil. Black soil: Deccan Maharashtra. Dunes: Rajasthan/Egypt.
Flora / Foliage	Rubber plantations, coconut palms, tea shrubs, conifers	Terraced tea: Munnar/Wayanad. Palms: Alappuzha. Pine/Fir: Swiss Alps / Kashmir.
Signage & Scripts	Malayalam rounded script, Devanagari, Kanji, Latin	Rounded script = Kerala. Bold top-line bar = Hindi belt. Kanji = Japan. Hangul = South Korea.

2. Technical Architecture: StreetCLIP, Residual Adapter & Multi-Scale TTA

Standard classification models fail in GeoGuessr because training a network from scratch on 200 places requires tens of thousands of labeled images. If you add place #201, you must retrain everything. Instead, our system utilizes a **Zero-Shot Vision-Language Foundation Model** coupled with a **Lightweight Residual Vision Adapter**.

INFERENCE PIPELINE ARCHITECTURE

Step 1: StreetCLIP Backbone (Frozen 1.7GB)
Pre-trained on millions of global street scenes, mapping images and natural language text into a shared 768-dim embedding space.

Step 2: Multi-Scale Test-Time Augmentation (TTA)
Instead of just resizing the image, we extract 3 distinct crops:
• **Full Scene (60% weight):** Captures global atmosphere, sky, weather, and panoramic context.
• **Lower Road Crop (25% weight):** Zooms into asphalt, lane markings, soil color, curb stones, and road width.
• **Center Horizon Crop (15% weight):** Isolates distant mountain ridges, tree lines, roof architecture, and utility poles.

Step 3: Residual Vision Adapter (clip_adapter.pt • 1.5MB)
A bottleneck MLP: [768 → 256 → GELU → 768] with identity residual mixing: **Output = 0.50 × Original + 0.50 × Residual**. This adapts visual features towards our location prompt vocabulary without suffering representation collapse.

Why We Use a Residual Adapter Instead of Full Fine-Tuning:

When full backpropagation is applied to the 1.7GB model on small custom datasets, the weights suffer **Catastrophic Representation Collapse**: the projection layers rotate towards the few fine-tuned samples, causing unrelated global images to falsely predict that single sample with 100% confidence. Our frozen backbone + residual adapter approach guarantees **100% accuracy on custom training samples** while completely preserving zero-shot generalization worldwide.

3. Active Learning & Strict Duplicate Prevention

When an AI model makes an incorrect guess, human feedback must correct it without breaking existing knowledge. Our system implements **Human-in-the-Loop Active Retraining** with 3 layers of defense:

1. Non-Destructive Visual Fingerprinting (corrections_cache.pt)

When a user corrects an image, its 768-dimensional normalized image vector is saved as a visual fingerprint. During subsequent inference, if a query photo matches an existing fingerprint with cosine similarity ≥ 0.92, a calibrated confidence boost is applied immediately to the corrected target. Unrelated images remain untouched.

2. Interactive Real-Time Search Bar

Under Section 3 in the Streamlit app, selecting **'No, choose from available options'** reveals an instant search bar. Teammates and users can type any keyword (e.g. *Kozhikode, Munich, Tokyo, Wayanad, France*) or filter by category (e.g. *Kerala - Micro-Region*). The app filters all 202+ locations in real time, displaying driving rules and GPS tags.

3. Multi-Layer Duplicate Prevention ('No Place Added Twice')

To ensure the database never accumulates redundant or duplicate locations:

- **Load-Time Pruning:** Upon application start, all locations are deduplicated by lowercase canonical name before building tensor matrices.
- **Live Registration Alerts:** If a user enters a place that already exists, the UI flashes a warning indicating that retraining will update the existing record in-place rather than creating a duplicate.
- **In-Place Vector Update:** In apply_user_correction(), existing places update their embeddings at their current index; only genuinely unique places trigger an append.
- **Safety Net Deduplication:** A final deduplication pass executes right before writing embeddings_cache.pt to disk.

4. The 4-Person Team Roles & Operational Playbook

To scale this project efficiently, work is divided across 4 specialized roles. Each person owns specific files, eliminating merge conflicts and ensuring clear accountability.

Role 1: Location Updater (Geographic Data Curator)

ROLE 1: LOCATION UPDATER

Primary Mission: Expand and curate the location database with accurate GPS coordinates, regional hierarchy, and rich GeoGuessr visual clues.
Owned File: locations.py • **Key Command:** python cache_embeddings.py
Step-by-Step Task:
1. Open locations.py and add new location dictionaries under the LOCATIONS list.
2. Ensure you include: name, lat, lon, country, region, category, driving_side, and prompt.
3. In the prompt string, describe visual cues: road lines, soil color, foliage, signage, and architecture.
4. Run python cache_embeddings.py to encode the multi-prompt embeddings into embeddings_cache.pt.
5. Run python validate_pipeline.py to confirm zero duplicates exist.

Role 2: Image Updater (Dataset & Street-View Collector)

ROLE 2: IMAGE UPDATER

Primary Mission: Collect, verify, and label high-resolution street view and landscape photos for training and testing.
Owned Directory: Imgs/ (and download_dataset.py) • **Key Command:** python validate_pipeline.py
Step-by-Step Task:
1. Check locations.py to see which locations lack sample photos in Imgs/.
2. Find wide-angle, clear outdoor street photos (via Wikimedia Commons, Unsplash, or local street shots).
3. **Strict Naming Convention:** Save as {Location_Name}_{Country}.jpg (e.g. Wayanad_Kerala_India.jpg).
4. **Quality Rules:** Avoid close-up portraits, macro shots of flowers, blurry photos, or extreme night shots.
5. Run python validate_pipeline.py to confirm all photos are valid, uncorrupted JPEGs.

Role 3: Model Trainer (Machine Learning Engineer)

ROLE 3: MODEL TRAINER

Primary Mission: Train the Residual Vision Adapter on newly added images and benchmark prediction accuracy.
Owned Files: train_adapter.py, test_accuracy.py, clip_adapter.pt
Key Commands: python train_adapter.py and python test_accuracy.py
Step-by-Step Task:
1. Pull the latest locations.py and Imgs/ added by Roles 1 and 2.
2. Run python train_adapter.py. The script loads all images in Imgs/, runs multi-scale TTA, computes cross-entropy loss against hard negative candidates, and trains the adapter for 35 epochs using AdamW & CosineAnnealing.
3. Run python test_accuracy.py to verify that Top-1 accuracy approaches 100%.
4. Commit the updated, lightweight clip_adapter.pt (~1.5 MB) to Git.

Role 4: App & Systems Developer (Full-Stack Engineer)

ROLE 4: APP & SYSTEMS DEVELOPER

Primary Mission: Maintain and upgrade the Streamlit web application, CLI predictor, maps integration, and UI features.
Owned Files: app.py, predict.py, requirements.txt • **Key Command:** streamlit run app.py
Step-by-Step Task:
1. Integrate new features into app.py (e.g. new satellite map layers, UI widgets, responsiveness).
2. Ensure external navigation links (Google Maps, Google Earth 3D, Street View) format coordinates accurately.
3. Keep predict.py in sync with app.py so CLI predictions match web app predictions.
4. Test the user experience thoroughly and verify that active retraining reruns smoothly.

5. Git & GitHub Workflow for Beginners (Zero-Conflict Strategy)

To collaborate smoothly without stepping on each other's toes, follow this golden rule: **Never push directly to the main branch**. Everyone works in their own branch and creates Pull Requests.

Step 1: Clone the Repo:

```
git clone https://github.com/mohammednihanvs-prog/geoguessr-ai.git
cd geoguessr-ai
```

Step 2: Create Your Role Branch:

```
# Role 1: git checkout -b data/locations
# Role 2: git checkout -b data/images
# Role 3: git checkout -b ml/training
# Role 4: git checkout -b feat/app-ui
```

Step 3: Run Pre-Flight Validation:

```
python validate_pipeline.py
```

Step 4: Stage, Commit & Push:

```
git add .
git commit -m "Descriptive explanation of your updates"
git push -u origin
```

Step 5: Open a Pull Request (PR):

```
Go to GitHub -> click 'Compare & Pull Request' -> request team review -> Merge into main.
```

6. Quality Gate: The Pre-Flight Automated Validation Script

To prevent broken code, missing files, or duplicate locations from ever reaching GitHub, we created an automated pre-flight tool: validate_pipeline.py.

Every teammate must run this command in their terminal before making a commit:

```
python validate_pipeline.py
```

What the Validation Script Tests Automatically:

- **1. Locations Integrity:** Verifies locations.py exists, ensures zero duplicate names exist, and confirms every entry has lat, lon, country, and category.
- **2. Image Dataset Health:** Scans all images in Imgs/ using PIL, ensuring files are valid and uncorrupted.
- **3. Weights & Caches:** Checks that embeddings_cache.pt and clip_adapter.pt are present and within valid size thresholds.
- **4. Code Syntax Compilation:** Runs py_compile across app.py, predict.py, train_adapter.py, and cache_embeddings.py to catch any Python syntax errors before runtime.

7. Troubleshooting & Frequently Asked Questions (FAQ)

Q: The AI keeps guessing the wrong place for my photo. How do I fix it?

Ans: Use Section 3 in the web app (Human-in-the-Loop Retraining). Select 'No, choose from available options', search for the correct spot, and click 'Retrain Model on this Image'. The app immediately saves a visual fingerprint with cosine similarity ≥ 0.92 and updates predictions instantly without catastrophic forgetting.

Q: Why shouldn't we commit fine_tuned_streetclip_backup.pt to GitHub?

Ans: That file is 1.71 GB. GitHub enforces a strict 100 MB per-file upload limit and will reject pushes containing it. Our .gitignore automatically ignores it. All team members use the pristine StreetCLIP backbone from Hugging Face and only track our lightweight clip_adapter.pt (1.5 MB), which fits easily into Git.

Q: What if two teammates edit locations.py at the same time?

Ans: Follow the Assembly Line order: Role 1 updates locations.py first and merges. Role 2 then pulls main before adding images. If a merge conflict ever happens, Git will highlight the conflicting lines with <<<<<<< HEAD. Keep both location dictionaries, run python validate_pipeline.py to ensure no duplicate names, and commit.

Q: How do we run the web app vs. the CLI tool?

Ans: For the interactive web interface: streamlit run app.py.
For terminal inference and heatmap generation: python predict.py.